POLITECNICO
MILANO 1863

Prova Finale

Progetto di Reti Logiche

Davide Petruzzi

21 ottobre 2025

Indice

1 Introduzione **2**
1.1 Obiettivi del Progetto 2
1.2 Specifiche generali 2
1.3 Interfaccia del componente 3
1.4 Specifiche del filtro differenziale 4

2 Architettura **5**
2.1 Segnali e variabili interne 5
2.2 Scelte progettuali 6
2.3 Descrizione macchina a stati 6
2.3.1 Stato IDLE 7
2.3.2 Stato READ_K_MSB 7
2.3.3 Stato READ_K_LSB 7
2.3.4 Stato READ_S_HEADER 7
2.3.5 Stato READ_C_HEADER 7
2.3.6 Stato READ_DATA 8
2.3.7 Stato ELABORATION_DATA 8
2.3.8 Stato WRITE_DATA 8
2.3.9 Stato WAIT_FOR_READ_DATA 8
2.3.10 Stato DONE 9

3 Risultati sperimentali **10**
3.1 Strumenti utilizzati 10
3.2 Report di sintesi 10
3.2.1 Timing report 10
3.2.2 Utilization report 10
3.3 Simulazioni behavioral 11
3.3.1 Primo testbench 11
3.3.2 Secondo testbench 12

4 Conclusioni **14**

1 Introduzione

1.1 Obiettivi del Progetto

Il progetto ha come obiettivo quello di implementare un modulo hardware, descritto in VHDL, che si interfaccia con una memoria RAM e che legge, un byte alla volta, un messaggio contenuto in essa costituito da un preambolo iniziale di 17 byte, e da una sequenza di K parole W, il cui valore, compreso tra -128 e +127, è rappresentato in complemento a 2, e applica un filtro differenziale e salva la sequenza di K parole R in memoria.

1.2 Specifiche generali

Il modulo deve essere in grado di gestire un messaggio di lunghezza variabile, di almeno 7 byte. L'indirizzo di partenza della memoria RAM è fornito dall'ingresso primario i_add. I primi 17 byte del messaggio contengono informazioni sul messaggio e su come elaborarlo. In particolare, i primi due byte K1 e K2 contengono la lunghezza del messaggio, il terzo byte S, serve per definire l'ordine del filtro differenziale, mentre i successivi 14 byte contengono i coefficienti del filtro differenziale. I primi 7 sono i coefficienti del filtro di ordine 3, i successivi 7 sono i coefficienti del filtro di ordine 5. Dopodiché inizia la sequenza di K parole W, che sono i dati da filtrare. La sequenza di K parole R, dove ogni parola costituisce il risultato di un'elaborazione, è memorizzata a partire dall'indirizzo $i_add + 17 + K$. Il modulo avvia l'elaborazione del messaggio quando un segnale di start (i_start) viene portato a 1 e termina quando il segnale di done (o_done) viene portato a 1. Prima del primo start verrà sempre dato un reset (i_rst). Di seguito due brevi schemi sull'interazione tra RAM e modulo (Figura 1.1) e sulla RAM (Figura 1.2):

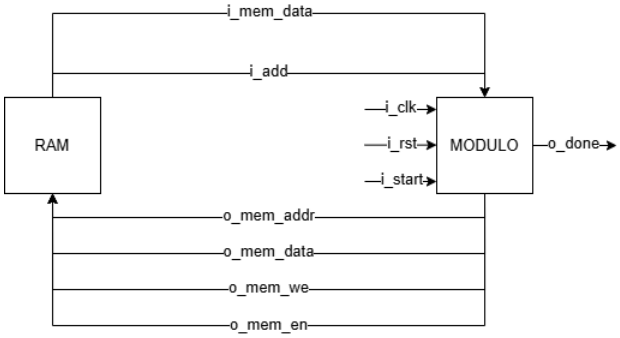

Figura 1.1: Schema della RAM collegata al modulo

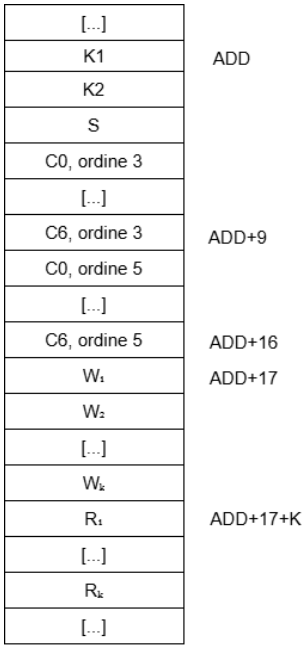

Figura 1.2: Schema della RAM

1.3 Interfaccia del componente

Il componente da descrivere possiede la seguente interfaccia:

```
entity project_reti_logiche is
    port (
        i_clk : in std_logic;
        i_rst : in std_logic;
        i_start : in std_logic;
        i_add : in std_logic_vector(15 downto 0);
        o_done : out std_logic;
        o_mem_addr : out std_logic_vector(15 downto 0);
        i_mem_data : in std_logic_vector(7 downto 0);
        o_mem_data : out std_logic_vector(7 downto 0);
        o_mem_we : out std_logic;
        o_mem_en : out std_logic
    );
end project_reti_logiche;
```

In particolare:

- *i_clk*: segnale di clock del sistema, viene generato dal testbench e ha un periodo di almeno 20 ns.
- *i_rst*: segnale di reset del sistema, viene generato dal testbench ed è asincrono.
- *i_start*: segnale di start dell'elaborazione del messaggio, generato dal testbench.
- *i_add*: indirizzo di partenza del messaggio in memoria, generato dal testbench.
- *o_done*: segnale che indica la fine dell'elaborazione del messaggio.
- *o_mem_addr*: segnale di uscita che manda l'indirizzo della memoria da leggere/scrivere.
- *i_mem_data*: segnale di ingresso contenente i dati letti dalla memoria.
- *o_mem_data*: segnale di uscita contenente i dati da scrivere nella memoria.
- *o_mem_we*: segnale di write enable della memoria, se a 1 la memoria è in scrittura, se a 0 è in lettura.
- *o_mem_en*: segnale di enable della memoria, se a 1 la memoria è abilitata, se a 0 è disabilitata.

1.4 Specifiche del filtro differenziale

Il filtro differenziale che deve essere implementato ha la seguente formula:

$$f'(i) = (1/n) * \sum_j C_j * f[j + i] \tag{1.1}$$

dove: j va da -I a +I, con I che vale 2 se il filtro è di ordine 3, e 3 se il filtro è di ordine 5. Il risultato del filtro deve essere normalizzato e saturato in modo da rimanere compreso tra -128 e +127. Per la normalizzazione, si usa n=12 per il filtro di ordine 3 e n=16 per il filtro di ordine 5, e viene approssimata nel seguente modo: 1/12 è approssimato con 1/16+ 1/64+1/256+1/1024, mentre 1/60 è approssimato con 1/64 + 1/1024. La saturazione viene effettuata in modo da mantenere il risultato compreso tra -128 e +127, se il risultato è maggiore di 127, viene saturato a 127, se è minore di -128, viene saturato a -128.

2 Architettura

2.1 Segnali e variabili interne

NOME	TIPO	DESCRIZIONE
`current_state`	`state_type`	Segnale che indica lo stato corrente.
`next_state`	`state_type`	Segnale che indica lo stato successivo.
`byte_counter`	`unsigned(15 downto 0)`	Contatore dei byte letti.
`addr_counter`	`unsigned(15 downto 0)`	Contatore degli indirizzi letti, che parte da *i_add*, fino ad arrivare a *i_add*+K+17.
`coefficient_counter`	`integer`	Contatore dei coefficienti letti, che parte da 0 e arriva a 6.
`K`	`std_logic_vector(15 downto 0)`	Segnale che contiene il valore di K, letto dalla memoria.
`S`	`std_logic`	Segnale che contiene il valore dell'ultimo bit di S, letto dalla memoria.
`C0, C1, ..., C6`	`signed(7 downto 0)`	Segnali che contengono i valori dei coefficienti letti dalla memoria
`data_sequence`	`data_array`	Array che contiene i dati letti dalla memoria, di dimensione 7. Ogni elemento è un *signed*(7 *downto* 0).
`filter_result`	`signed(17 downto 0)`	Segnale che contiene il risultato del filtro differenziale.
`result`	`signed(17 downto 0)`	Variabile interna al processo *next_state_and_output_logic* che contiene il risultato normalizzato e saturato.
`result_address`	`unsigned(15 downto 0)`	Variabile interna al processo *next_state_and_output_logic*, che contiene l'indirizzo di memoria in cui scrivere il risultato.
`shift1, ..., shift4`	`signed(17 downto 0)`	Variabili interne al processo *next_state_and_output_logic*, che contengono il risultato shiftato a destra.

Tabella 2.1: Segnali e variabili interne del modulo.

Nella Tabella 2.1 sono riportati i segnali e le variabili interne del modulo, con il rispettivo nome, tipo e descrizione. Non sono stati riportati i segnali interni next_..., in quanto sono segnali equivalenti concettualmente a quelli descritti, ma che vengono utilizzati per calcolare i valori prossimi dei registri, senza modificarli immediatamente durante il ciclo di clock. In questo modo si possono aggiornare i registri in modo sincrono, al successivo fronte di

clock, e si evitano modifiche simultanee che potrebbero causare problemi di sincronizzazione o latch indesiderati.

2.2 Scelte progettuali

Per motivi di semplicità e compattezza, si è scelto di non adottare una struttura modulare, mantenendo tutta la logica all'interno di un'unica entity con processi ben distinti, seguendo un approccio di tipo *behavioral* e utilizzando una macchina a stati finiti. Si è scelto di creare tre processi, di cui due sincroni: *state_reg*, sensibile al clock e al reset, per gestire le transizioni di stato della macchina a stati e l'aggiornamento dei segnali interni, e *o_done_logic*, per la gestione del segnale di *o_done*; uno combinatorio, *next_state_and_output_logic*, per codificare sia la funzione di stato prossimo che la funzione di uscita per i segnali di memoria. Inizialmente si era pensato di implementare il modulo mediante solo i primi due processi, ma si è deciso di aggiungere un terzo processo per evitare la creazione di latch dovuti ad assegnamenti mancanti al segnale *o_done*, dopo che questi erano stati evidenziati da una prima sintesi.

2.3 Descrizione macchina a stati

La macchina a stati, rappresentata in Figura 2.1, pag. 9, è composta da 10 stati.

- IDLE
- READ_K_MSB
- READ_K_LSB
- READ_S_HEADER
- READ_C_HEADER
- READ_DATA
- ELABORATION_DATA
- WRITE_DATA
- WAIT_FOR_READ_DATA
- DONE

I 4 stati necessari per la lettura dell'header, `READ_K_MSB`, `READ_K_LSB`, `READ_S_HEADER` e `READ_C_HEADER`, potevano essere accorpati in un unico stato, ma si è preferito mantenerli separati per una maggiore chiarezza del codice e della logica di funzionamento.

2.3.1 Stato IDLE

È lo stato iniziale: in questo stato il modulo è in attesa di un segnale di start (*i_start*), che se attivo consente il passaggio allo stato `READ_K_MSB` e richiede l'accesso in lettura alla memoria, per iniziare a leggere l'header del messaggio. Se il *i_rst* è 1, il modulo torna sempre in IDLE. Prima del primo segnale di start, il modulo aspetta sempre un segnale di reset.

2.3.2 Stato READ_K_MSB

In questo stato il modulo legge il byte più significativo di K, che è memorizzato in *i_mem_data*, e lo salva negli 8 bit più significativi del segnale *next_K*. Il modulo poi passa allo stato `READ_K_LSB`, richiedendo già l'accesso in lettura alla memoria per il byte meno significativo di K.

2.3.3 Stato READ_K_LSB

In questo stato il modulo legge il byte meno significativo di K, che è memorizzato in *i_mem_data*, e lo salva negli 8 bit meno significativi del segnale *next_K*. Il modulo poi passa allo stato `READ_S_HEADER`, richiedendo già l'accesso in lettura alla memoria per il byte S.

2.3.4 Stato READ_S_HEADER

In questo stato il modulo legge il byte S, che è memorizzato in *i_mem_data*, e salva l'ultimo bit nel segnale *next_S*. Il modulo poi passa allo stato `READ_C_HEADER`, richiedendo già l'accesso in lettura alla memoria. Inoltre, sulla base del valore dell'ultimo bit del byte appena letto, verifica se il filtro da applicare è di ordine 3 o 5 e, a seconda del tipo di fitro, calcola il prossimo indirizzo di memoria da leggere.

2.3.5 Stato READ_C_HEADER

In questo stato il modulo legge i coefficienti del filtro differenziale, uno alla volta, e li salva nei segnali *next_Ci*. Mantenendo il conteggio con *coefficient_counter*, una volta memorizzato l'ultimo coefficiente, il modulo passa

allo stato `READ_DATA`, richiedendo già l'accesso in lettura alla memoria per il primo byte di dati, e inizializzando a 0 le prime quattro posizioni dell'array *data_sequence*.

2.3.6 Stato READ_DATA

In questo stato il modulo legge i dati, uno alla volta, e li salva nell'array *data_sequence*. In particolare sono gestiti i casi degli estremi della sequenza, con le condizioni $byte_counter <= 3$, per porre nella sequenza degli 0 nelle posizioni antecedenti a quella centrale (utili per l'elaborazione dei primi byte), e $byte_counter > unsigned(K)$ per porre nella sequenza degli 0 nelle posizioni successive alla posizione centrale (utili per l'elaborazione degli ultimi byte).
Il modulo poi passa allo stato `ELABORATION_DATA`, a meno che non sono stati elaborati tutti i dati, in quel caso passa allo stato `DONE`.

2.3.7 Stato ELABORATION_DATA

In questo stato il modulo elabora il dato in posizione centrale dell'array *data_sequence*, applicando il filtro differenziale e salvando in *next_filter_result* il risultato dell'elaborazione.
Poi il modulo passa allo stato `WRITE_DATA`.

2.3.8 Stato WRITE_DATA

In questo stato il modulo esegue normalizzazione e saturazione del risultato dell'elaborazione, e lo salva in *o_mem_data*. Il modulo poi passa allo stato `WAIT_FOR_READ_DATA`, richiedendo l'accesso in scrittura alla memoria per il byte di risultato.

2.3.9 Stato WAIT_FOR_READ_DATA

In questo stato il modulo richiede l'accesso in lettura alla memoria per il prossimo byte di dati, e passa allo stato `READ_DATA`. È stato aggiunto questo stato per evitare conflitti sul segnale *i_mem_data*, in quanto una richiesta immediata di lettura della memoria faceva sì che venisse caricato su *i_mem_data* il risultato appena scritto in memoria, e non il prossimo byte di dati da elaborare.

2.3.10 Stato DONE

In questo stato viene portato alto il segnale di uscita *o_done*, per indicare la fine dell'elaborazione del messaggio. Il modulo resta in questo stato finché non viene portato a 0 il segnale di start, in quel caso torna nello stato IDLE, dove il segnale *o_done* viene riportato a 0.

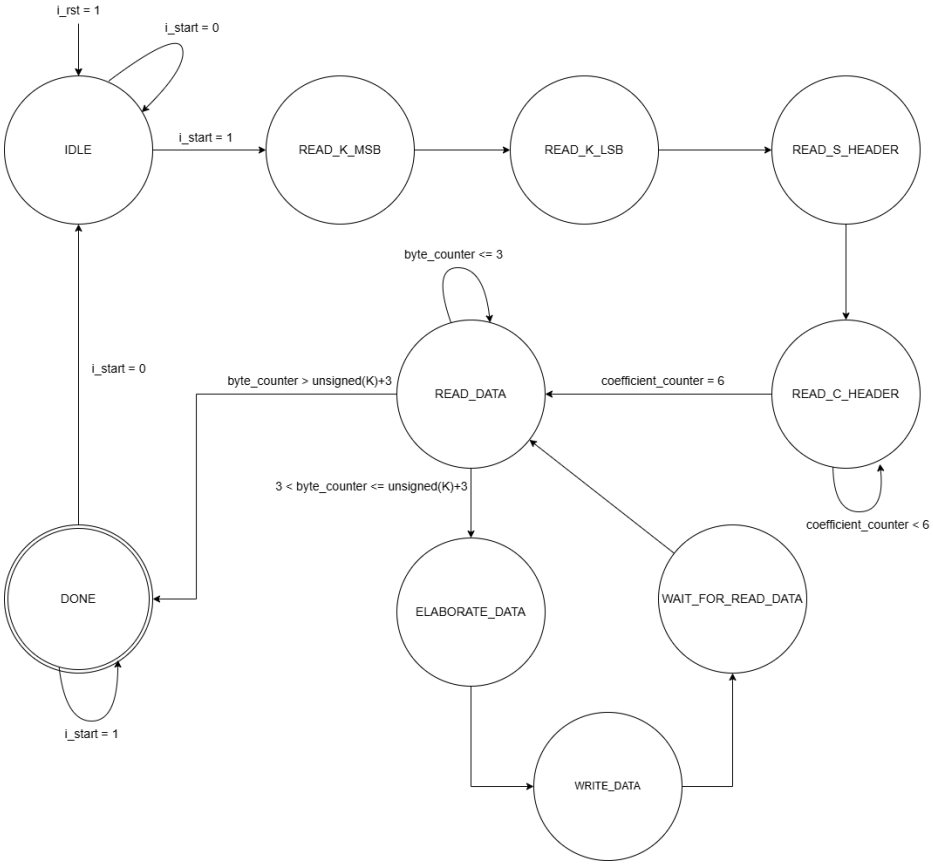

Figura 2.1: Diagramma della macchina a stati finiti

3 Risultati sperimentali

3.1 Strumenti utilizzati

Per effettuare la sintesi e le simulazioni del progetto, è stato utilizzato il tool Vivado Synthesis 2016, parte di Xilinx Vivado Design Suite 2016.1. La sintesi è stata effettuata su un dispositivo FPGA Artix-7 xc7a100tcsg324-3.

3.2 Report di sintesi

3.2.1 Timing report

Il **timing report** (Figura 3.1) è stato generato dopo la sintesi del progetto, e in particolare evidenzia un WNS (*Worst Negative Slack*) di 11,138 ns rispetto ad un periodo di clock di 20 ns. Questo significa che il modulo è in grado di funzionare correttamente con un clock di 20 ns, e che non ci sono problemi di timing, in quanto il percorso critico più lungo del circuito è di 8,862 ns, che è ben inferiore al periodo di clock.

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	11,138 ns	Worst Hold Slack (WHS):	0,152 ns	Worst Pulse Width Slack (WPWS):	9,500 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	292	Total Number of Endpoints:	292	Total Number of Endpoints:	184

Figura 3.1: Timing report del progetto

3.2.2 Utilization report

In Figura 3.2 e 3.3 vi sono i report di sintesi che evidenziano l'utilizzo delle risorse del dispositivo FPGA. Sono state utilizzate 841 LUT (*Look Up Table*), e 183 *Flip Flop*. Come si vede dal report, tutti gli *Slice Registers* sono stati utilizzati come *Flip Flop*, per cui non ci sono *Latch*.

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	841	0	63400	1.33
LUT as Logic	841	0	63400	1.33
LUT as Memory	0	0	19000	0.00
Slice Registers	183	0	126800	0.14
Register as Flip Flop	183	0	126800	0.14
Register as Latch	0	0	126800	0.00
F7 Muxes	0	0	31700	0.00
F8 Muxes	0	0	15850	0.00

Figura 3.2: Tabella risorse

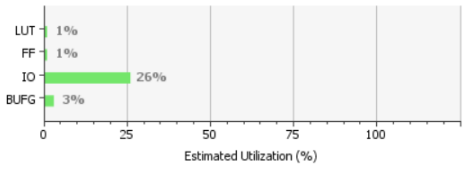

Figura 3.3: Grafico dell'utilizzo delle risorse

3.3 Simulazioni behavioral

Per verificare il corretto funzionamento del modulo, sono state effettuate delle simulazioni *behavioral* e *post-synthesis functional*, utilizzando due testbench diversi. Il secondo testbench, in particolare, sottopone il modulo a diversi casi di test per verificarne il corretto funzionamento in diverse condizioni, per cui questi due testbench sono stati reputati esaustivi.

3.3.1 Primo testbench

Il primo testbench, **tb2425**, è quello fornito dal docente, e contiene un'unica sequenza di dati che testa solo il filtro di ordine 3, in quanto l'ultimo bit del byte S è 0.
Le simulazioni *behavioral* e *post-synthesis* del testbench tb2425 terminano in 3230ns. Nella simulazione post-sintesi si può vedere come i segnali *current_state* e *next_state* vengono rappresentati in binario, con 4 bit, e come *data_sequence* viene spezzato in 7 registri, ognuno di 8 bit, per rappresentare i dati letti dalla memoria.
Qui sotto sono riportati i risultati della simulazione *behavioral* e *post-synthesis functional* del testbench **tb2425**, fotografati all'istante finale, dai quali si può notare come *data_sequence* contenga in posizione centrale l'ultimo dato elaborato e tutti 0 nelle posizioni successive. Lo stato passa correttamente a DONE, e, quando il segnale di start viene portato a 0, prima *next_state* e poi anche *current_state* tornano a IDLE. Nell'ultimo istante di simulazione, il segnale *o_done* viene portato a 0.

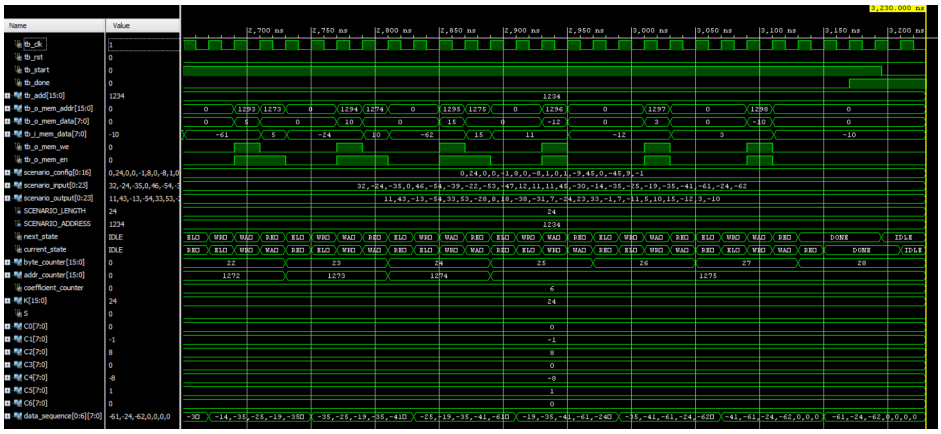

Figura 3.4: Simulazione behavioral del testbench tb2425

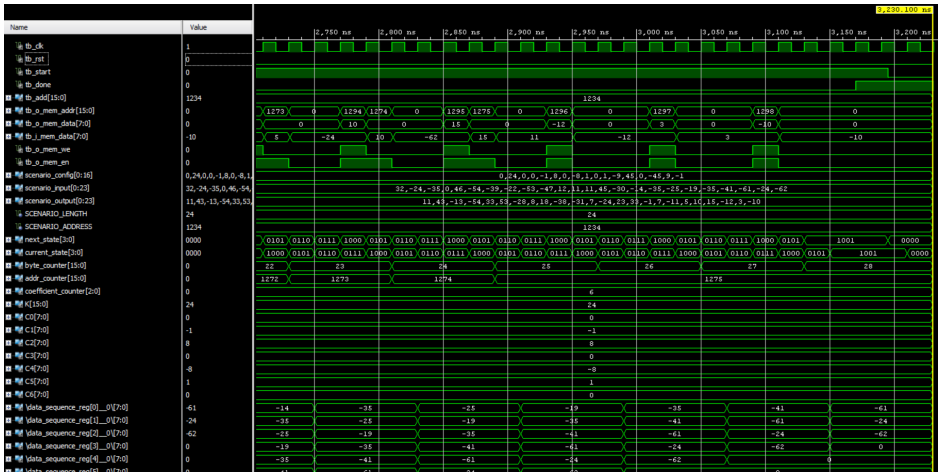

Figura 3.5: Simulazione post-sintesi del testbench tb2425

3.3.2 Secondo testbench

Il secondo testbench provato, **testbench_mark2**, testa il modulo combinando diversi test. Sono infatti presenti ben sei scenari diversi in cui:

- sono testati sia il filtro di ordine 3, che di ordine 5;
- vi sono diversi reset, che permettono di testare il corretto funzionamento del modulo in caso di reset durante, o al termine dell'elaborazione;
- vi sono sequenze di dati di lunghezza variabile, sia sequenze molto lunghe, che sequenze minime con K=7;
- assicura che il modulo funzioni correttamente anche in caso il primo e ultimo coefficiente sono diversi da zero;
- verifica che i risultati siano troncati correttamente, testando anche i valori massimi ottenibili prima della saturazione negli scenari 5 e 6.

Il test viene passato in 2552990 ns, e il modulo termina correttamente l'elaborazione del messaggio, per tutti e sei gli scenari.
Di seguito sono riportati i risultati delle simulazioni *behavioral* e *post-synthesis functional* del testbench **testbench_mark2**, fotografati all'istante finale, prendendo in causa solo l'ultimo scenario, che testa il filtro di ordine 5, con una sequenza di 7 byte, tutti uguali a -128, e con i coefficienti del filtro di ordine 5 tutti uguali a -128, analizzando il calcolo più grande. L'output atteso, ed ottenuto, è una sequenza di 7 byte, tutti uguali a 127.

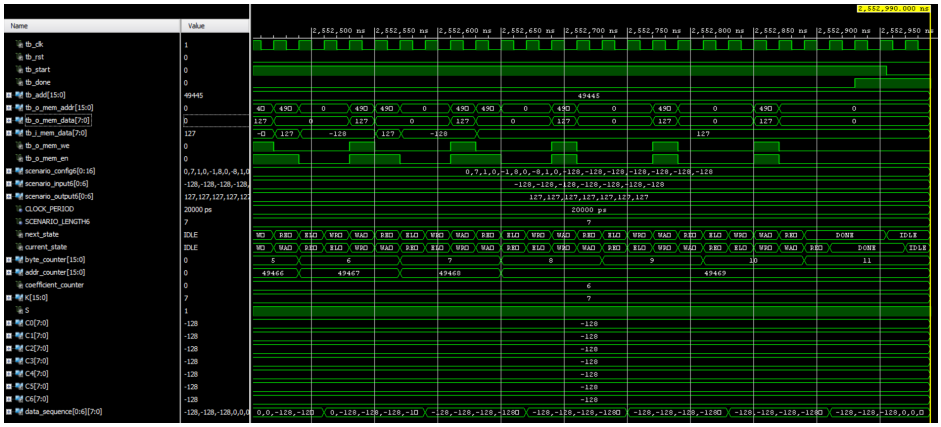

Figura 3.6: Simulazione behavioral del testbench testbench_mark2

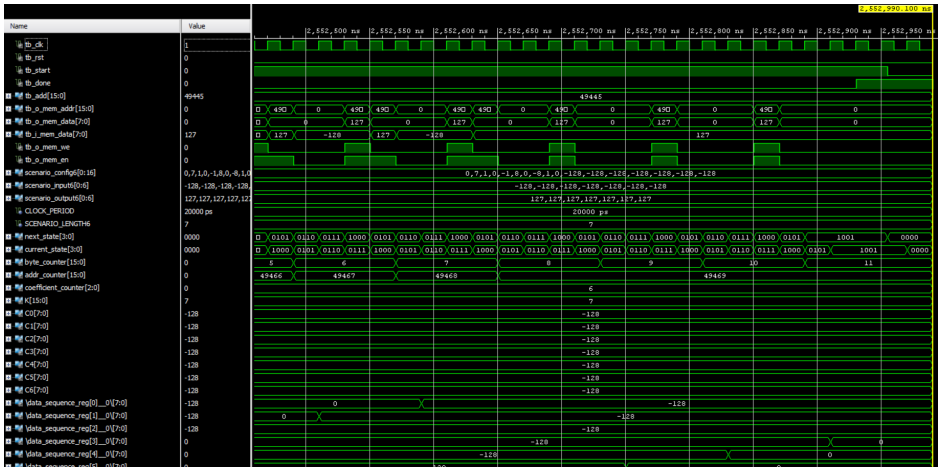

Figura 3.7: Simulazione post-sintesi del testbench testbench_mark2

In alcuni casi, durante la simulazione post-sintesi functional in Vivado, il test non termina automaticamente con un messaggio esplicito nella console Tcl, pur avendo completato correttamente l'esecuzione. Tale comportamento non indica necessariamente un malfunzionamento del progetto, ma è legato alla natura della simulazione post-sintesi, più lenta e meno ottimizzata rispetto alla simulazione comportamentale. Tuttavia, l'analisi della waveform e dei segnali interni conferma il corretto completamento e il rispetto del comportamento atteso, come evidenziato anche dalle immagini dell'ultimo scenario. Come spiegato in precedenza, negli altri scenari sono presi in esame altri casi limite, come il conto massimo con termini positivi (coefficienti e dati posti a 127), sequenza di dati molto lunga e, in particolare per il filtro di ordine 3, primo e ultimo coefficiente diversi da 0.

4 Conclusioni

Il modulo funziona correttamente con tutti i test provati, sia in *Behavioral*, che in *Post-Synthesis Functional*. Come già spiegato nel capitolo 2, avrei potuto ridurre il numero di stati della FSM, accorpando gli stati di lettura dell'header in uno solo, ma ho preferito mantenerli separati per una maggiore chiarezza della logica di funzionamento, e per evitare di dover gestire un numero maggiore di condizioni per la transizione tra gli stati.
Le principali problematiche riscontrate durante la progettazione, sono state evidenziate già dalla simulazione behavioral, e riguardavano soprattutto l'aggiornamento dei segnali interni, che spesso non avveniva correttamente. In fase di sintesi i problemi sono stati minori e riguardavano principalmente qualche assegnamento mancante. Una volta risolti questi problemi e qualche errore di sintassi iniziale, il modulo ha funzionato adeguatamente. Sono state effettuate due ottimizzazioni in particolare: una riguardante la dimensione del risultato intermedio del filtro, che è di 18 bit, ossia il numero minimo di bit necessari per rappresentare il risultato massimo del filtro prima della normalizzazione, e una riguardante il segnale S, che inizialmente era stato pensato con dimensione di un byte, per poi essere ridotto ad un solo bit, quello necessario per determinare l'ordine del filtro.